Open in app ↗ Sign up Sign in

Medium Search Write

Dev Genius

Just Built A Two-Layer AI System That Trades Polymarket and Kalshi While I Sleep: Here’s The Complete Technical Blueprint (And Why Most AI Agent Setups Are Broken)

Ezekiel Njuguna Follow 17 min read · Mar 3, 2026

11

Photo by Mathieu Stern on Unsplash

I need to tell you something that fundamentally changed how I think about AI agents and automation: most people building "AI agents" are actually just building expensive chat interfaces with file access. They're not building real automation. They're building glorified assistants that still require constant human supervision, manual file copying, and prayer that the model doesn't hallucinate commands that delete your important data.

What I'm about to describe is different. This is a proper two-layer architecture where a "brain" AI plans and supervises while a "hands" AI executes locally with deterministic task tracking, crash recovery, and real security boundaries. This isn't a demo that works once. This is a system designed to run unattended for days while paper trading prediction markets, discovering arbitrage opportunities, and only asking for human input when genuinely stuck.

The core insight that makes this work: you need a shared workspace folder that acts as a communication bus between the brain and hands, with explicit task files, state machines, and atomic operations. No magic. No hoping the AI "remembers" context. Just files, protocols, and processes that work even when everything crashes.

I'm going to walk through exactly how to build this, what technical corrections are essential to make it actually work, why most simplified guides leave out the critical details that cause systems to fail in production, and how to design the prediction market trading layer so it's mathematically sound rather than just probabilistic guessing disguised as arbitrage.

Why The Brain And Hands Architecture Actually Matters

The fundamental problem with most AI automation attempts is they try to make one model do everything. Planning, execution, file manipulation, external API calls, error recovery. This creates a mess where the model is constantly context-switching between high-level strategy and low-level implementation details.

The brain and hands separation solves this by having clear responsibilities:

The Brain (Supervisory AI):

- Writes structured task specifications into the shared workspace
- Reads results and execution logs
- Decides whether to retry, refine, or escalate based on outcomes
- Answers questions that require judgment about scope, priority, or risk

- Never directly executes destructive actions or touches production systems

The Hands (Execution AI):

- Picks up tasks from the workspace folder autonomously
- Executes exactly as specified with explicit tool constraints
- Writes outputs to predetermined locations
- Asks specific questions when blocked (not vague "what should I do")
- Maintains execution logs and state for crash recovery

This isn't just cleaner conceptually. It's operationally essential because the brain can be expensive (GPT-4, Claude Opus) while the hands can be cheap or free (local models via Ollama). You pay for intelligence where it matters and run execution loops at near-zero marginal cost.

The Technical Corrections Nobody Tells You About

Before going into implementation, I need to correct several things you'll see in simplified guides that will cause your system to fail.

Desktop Versus Browser Is Not The Dividing Line

Most guides say "use Claude Desktop for local access, use ChatGPT for cloud." This is wrong. The actual dividing line is whether your client supports the Model Context Protocol (MCP) for local server connections.

MCP is an open protocol designed to connect LLM applications to tools and external context. Which client you use determines how easily you can connect tools and local resources.

For Claude Desktop, MCP is specifically documented for connecting local MCP servers including filesystem access by editing a local config file. Actions require explicit user approval per operation. This makes shared-folder orchestration straightforward because Claude Desktop natively supports local MCP servers without requiring any tunneling or remote hosting.

For ChatGPT, MCP support exists through "developer mode" or apps, but OpenAI's documentation emphasizes connecting to MCP servers that are reachable over HTTPS for testing. This typically means you'll end up running an MCP server locally and exposing it via a tunnel like ngrok or Cloudflare Tunnel, or running the MCP server remotely. Either way, the "all local, no exposure" premise changes.

The practical consequence: if you want purely local operation with minimal glue, Claude Desktop with local MCP servers is the straightforward path. ChatGPT requires additional infrastructure decisions.

"Unlimited Questions For $20" Is A Dangerous Assumption

Plans marketed as "more" or "unlimited" still have usage limits, fallbacks, or fair-use policies. OpenAI's Help Center documents per-plan message limits for GPT-5.2 usage in ChatGPT (for example, Plus tier has caps with fallback behavior). Similarly, Anthropic positions Pro as "more usage" rather than unconditional unlimited.

The realistic engineering claim: a flat subscription dramatically reduces marginal cost per iteration, but "infinite" is not a dependable requirement you can architect around. Design your system to handle rate limits and fallbacks gracefully.

Free Local Agents Still Have Real Constraints

Running local models via Ollama is free in the "no token bill" sense, but context length and performance are constrained by RAM/VRAM and configuration. Ollama's documentation explicitly notes that effective context defaults vary by available VRAM, and that large-context tasks like agents, coding, and web search should be configured at high context sizes, which has hardware implications.

You cannot just "pull a 64k context model" and expect it to work. A model must support that context length, you must configure the runtime to use that context length (the num_ctx parameter), and your hardware must survive that context length without massive performance degradation.

For autonomous execution, you often get better throughput by using moderate context windows (8k to 32k) for repetitive tool runs and relying on retrieval (reading only needed files) instead of stuffing everything into context each cycle.

Probabilistic Correlations Are Not Arbitrage

This is the most important correction for the trading portion. Relationships like "rain leads to park cancellations" are not hard constraints. They're probabilistic correlations. Systems that treat probabilistic correlations as "guaranteed wins" are doing statistical speculation, not mathematical arbitrage.

If you want actual arbitrage with math-backed guarantees, you must focus on identities and inequalities that must hold by definition. For example, in binary markets, YES plus NO must equal $1 at settlement (portfolio payoff identity). In categorical markets with mutually exclusive outcomes, exactly one outcome resolves true (partition constraint).

These are logical requirements, not empirical patterns. And even with logical constraints, you still must incorporate fees, spreads, and execution risk or your "arbitrage" disappears in practice.

Setting Up The Brain Layer With MCP And Filesystem Access

The practical requirement for your brain is simple: it must reliably read and write to the shared workspace without manual copy-paste. Otherwise you haven't actually removed the human bottleneck.

Recommended Brain Configuration

The most direct implementation uses an MCP client that natively supports local MCP servers and filesystem tooling. The MCP documentation explicitly walks through connecting local MCP servers using Claude Desktop as the reference example, including the filesystem server setup and per-action approval system.

This pushes toward using Claude Desktop when your goal is local, file-based orchestration with minimal infrastructure glue.

Filesystem MCP: Exact Configuration And Why It Matters

The MCP "connect local servers" guide describes the Claude Desktop config file locations on macOS and Windows, and provides a JSON structure that launches the filesystem server via npx and explicitly restricts which directories are accessible.

This matters enormously because you should only grant access to directories you intend to operate in — your ai-workspace folder and nothing else. The MCP documentation explicitly warns that the server runs with your user permissions, so your authorization scope is your actual safety boundary.

The configuration looks like this conceptually:

You specify which directories the filesystem MCP server can access. The brain can then read and write files in those directories with explicit approval prompts for each operation. This prevents the brain from accidentally touching system files, personal documents, or anything outside the designated workspace.

Engineering The Brain As A Supervisor, Not A Doer

To make the system stable and maintainable, the brain's job should be strictly constrained:

Write structured task specifications into the workspace using a defined format. Read results and execution logs that the hands produce. Decide whether to retry with modifications, refine the approach, or escalate to human judgment. Answer questions that require scope decisions, priority calls, or risk assessment.

Critically, avoid letting the brain issue direct destructive actions like deleting directories, moving significant file trees, or rotating credentials. MCP's explicit approvals help enforce this, but you still want a clear policy: approvals are only for expected changes within the designated workspace.

The brain should think of itself as writing work orders and reading completion reports, not directly manipulating the production environment.

Setting Up The Hands Layer With Ollama And OpenClaw

The hands layer has two distinct responsibilities that work together: execution runtime (commands, file operations, running code) and agent loop (picking up tasks, tracking state, asking questions, writing results).

Local Model Runtime With Ollama

Ollama is a local inference runtime with official installers for macOS, Windows, and Linux. Installation on Linux is literally one command. Ollama's documentation covers context length fundamentals and how defaults vary with available VRAM, plus the num_ctx Modelfile parameter to configure desired context size for specific model builds.

The important engineering reality: you must explicitly configure context length based on your hardware capabilities and task requirements. Pulling a model doesn't automatically give you maximum context. You configure it, and then your hardware either handles it or slows to a crawl.

For an autonomous executor running repetitive tasks, moderate context windows (8k to 32k) often provide better throughput than trying to stuff 64k tokens into every operation. Use retrieval and file reading instead of holding everything in context.

Agent Runtime With OpenClaw

OpenClaw's documentation explicitly defines an "agent workspace" concept described as "the agent's home." The default workspace is located at ~/.openclaw/workspace, and it's the only working directory used for file tools and workspace context. It's deliberately separate from ~/.openclaw/ which holds config, credentials, and session data.

This maps perfectly onto the brain-hands shared folder concept. The simplest implementation:

Make your shared directory be the OpenClaw workspace (or a designated subfolder within it). Give the brain filesystem MCP access to that same

directory. Now both the brain and hands are reading and writing to the same location with clear boundaries.

OpenClaw supports configuring the workspace location in ~/.openclaw/openclaw.json, so you can point it wherever makes sense for your setup.

Heartbeat As Your Autonomy Loop

OpenClaw has a first-class "heartbeat" feature that's exactly what you need for autonomous operation. The heartbeat documentation states:

Default cadence is 30 minutes (with some variations based on authentication mode). The agent reads a file called HEARTBEAT.md if it exists and follows instructions in it. The default prompt behavior includes returning HEARTBEAT_OK if nothing requires attention.

This is the correct architectural place to encode your "check tasks folder, claim work, execute, write results, ask questions when stuck" logic because it's repeatable and already part of the runtime. You're not fighting against the framework or implementing your own polling loop.

A Safer Hands Deployment Pattern

If you intend this to run unattended for extended periods, resist the temptation to run it with full permissions on your primary personal machine.

OpenClaw's documentation describes the workspace as the default working directory but explicitly notes it's not a hard sandbox unless sandboxing is enabled. Absolute paths can still access files elsewhere. Additionally, there

have been real-world security concerns around unvetted OpenClaw "skills" distributing malware and misconfiguration leading to breaches.

Your safest production posture:

Run OpenClaw under a dedicated OS user account with minimal permissions, OR run it in a VM or container with explicit mounts only to your intended workspace directories, OR run tool execution in a containerized bridge pattern where execution happens in isolated containers.

This dramatically reduces blast radius if something goes wrong or if a component gets compromised.

The Shared Workspace Protocol That Makes Everything Reliable

A shared folder only becomes a real communication bus when you define task format, state machine, idempotency rules, and crash-safe writes. This section is the difference between a demo that works once and a system you can leave running.

Directory Structure That Works With Both MCP And OpenClaw

Because OpenClaw's workspace is explicitly the agent's home and is intended to contain instruction files like AGENTS.md, TOOLS.md, and HEARTBEAT.md, you should structure your shared workspace with the coordination protocol in designated subfolders.

A robust folder bus layout:

tasks/new/ — Brain writes task files here **tasks/in_progress/** — Hands atomically moves tasks here to claim them **tasks/done/** and **tasks/failed/** —

Hands archives completed tasks with status **results/** — Hands writes execution outputs here, brain reads them **brain_inbox/** — Hands writes questions here, brain provides answers **logs/** — Append-only execution logs for auditability **state/** — Lightweight JSON state files for resumability after crashes

Get Ezekiel Njuguna's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

This structure keeps high-churn files separate from the instruction root and prevents the agent from repeatedly re-reading bulky logs or archived tasks.

Task File Specification: Make It Machine-Checkable

Writing a markdown file that says "do X" works until you need retries, partial completion tracking, or concurrent tasks. You need structure.

Recommended task file format:

File type: JSON (not markdown, despite being more human-readable) **Naming convention:** YYYYMMDD-HHMMSSZ__shortslug__taskid.task.json **Atomic write rule for brain:** Write to .tmp file, then atomically rename to .task.json **Claim rule for hands:** Atomically rename from tasks/new/ to tasks/in_progress/

Minimum required fields in every task file:

task_id (UUID for unique identification) created_at (ISO8601 timestamp) priority (integer for sorting) intent (enum: code, data_pull, analyze, trade_sim, trade_live, ops) inputs (files, URLs, parameters needed) acceptance_criteria (list of verifiable conditions for completion) constraints (time budget, max network calls, security restrictions) deliverables (expected output filenames and formats) safety (allowed tool groups, permitted directories)

Why this structure matters: OpenClaw can be configured with tool allow and deny lists. Your task specification should align with the tools you actually permit for each task type. A data pull task might allow HTTP tools but not filesystem writes. A code execution task might allow filesystem but not network access.

HEARTBEAT.md Should Be Short And Act Like A Scheduler

OpenClaw's workspace documentation explicitly advises keeping heartbeat content minimal to avoid token waste and processing overhead. A production-grade heartbeat loop is therefore deterministic and concise:

If any tasks/new/*.task.json exists, claim exactly one by atomic rename to in_progress. Execute according to the task specification, limiting tool usage to the task's explicitly allowed tools. If blocked or unclear, write one specific question file to brain_inbox/ referencing the task_id. If completed successfully, write all deliverables to results/ and move task to done/. If execution fails, write task to failed/ with detailed error report and reproduction steps.

OpenClaw supports configuring heartbeat cadence in the config file and includes a default prompt that reads HEARTBEAT.md and returns

HEARTBEAT_OK when no work is pending. Your heartbeat file should encode the above logic concisely, not dump pages of instructions.

Logging And Auditability Are Not Optional

A system that trades, even paper trades, must be fully reconstructible after the fact for debugging, auditing, and learning.

OpenClaw's security documentation mentions that session transcripts are stored locally under ~/.openclaw/agents/agentId/sessions/*.jsonl and that filesystem access is the trust boundary for what's exposed.

Beyond OpenClaw's built-in session logs, you should implement:

Append-only execution logs per task (logs/taskid.log). Structured event logs (logs/events.jsonl) recording every significant action: data pull started, signal detected, simulated trade placed, fill assumption applied, profit and loss marked, error encountered, retry attempted. Immutable config snapshots per run (copy openclaw.json and runtime settings hashes into state/ before each run).

This makes "paper trading for 72 hours unattended" actually meaningful because you can validate every assumption, replay every decision, and prove what happened.

Prediction Market System Design For Real Trading

This section is technical engineering, not financial advice. The goal is designing a system that can paper trade deterministically defined constraint violations and only later consider live execution under strict risk controls.

Market Data Layer: Treat It As An Exchange Adapter Problem

Implement separate adapters for each venue even if you start with just one. This prevents your core logic from becoming tightly coupled to one platform's API quirks.

For Polymarket, the documentation states it uses a central limit order book (CLOB), recommends WebSocket API for live order book updates rather than polling, and uses limit orders (market-style execution is achieved via marketable limit orders). Polymarket's trading overview describes a hybrid structure with off-chain matching and on-chain settlement on Polygon, with audited exchange contracts.

For Kalshi, the API documentation covers market data and trade execution, and explicitly states that using the API means agreeing to be bound by the developer agreement. Kalshi also provides a demo environment with mock funds and a published demo API root URL for testing.

Your Layer 1 output should be a normalized internal representation regardless of venue:

market_id, venue, contract_type (binary or categorical), outcomes array, resolution_time, rules_hash. Order book snapshots with best bid/ask, depth ladder, timestamp. Recent trades with last price and volume. Fee structure and tick size for the specific market.

Fees are absolutely not optional. Polymarket's fee documentation describes that most markets are fee-free but some categories have taker fees. You must ingest the actual fee regime into your simulator or your profit calculations will be systematically wrong.

Constraint Detection: Restrict To Hard Constraints First

If you want mathematical certainty rather than probabilistic speculation, your constraint graph must be built from rules that are logically required, not empirically correlated.

Start with a small library of hard constraint templates:

Complement constraint (binary markets): YES plus NO equals $1 at settlement payoff level. This is a portfolio payoff identity. Trade prices can deviate due to spreads and fees, but the settlement constraint is absolute.

Partition constraint (categorical markets): Exactly one of N mutually exclusive outcomes resolves true. The sum of probabilities across all outcomes must equal 1.

Subset constraint: The probability of A and B occurring together cannot exceed the probability of A occurring alone. This is a dominance inequality that must hold.

Mutual exclusion: If two outcomes cannot both be true, P(A) plus P(B) must be less than or equal to 1.

Implication constraints: If market rules guarantee that A implies B, then feasible joint distributions are constrained accordingly.

These are mathematical requirements, not empirical patterns observed in data. This is the foundation of genuine arbitrage detection.

Arbitrage Math Layer: What The Optimization Actually Does

In prediction market theory, pricing rules and market maker mechanisms can be formally expressed using concepts like Bregman divergences,

marginal polytopes, and convex optimization methods. This sounds abstract but has a concrete engineering translation.

Step A: Define a constraint set over states and outcomes, typically as linear inequalities.

Step B: Given observed market prices (or implied probabilities), compute whether they're consistent with any feasible distribution under those constraints. This is a no-arbitrage consistency check.

Step C: If prices are inconsistent with constraints, compute a portfolio (a set of positions across related markets) whose worst-case payoff dominates its cost under your execution model including fees, spreads, and fill probability.

In exchange-traded settings with bid-ask spreads and partial fills, no-arbitrage becomes an LP-style dominance problem: portfolios that guarantee nonnegative payoffs across all possible outcomes must not have negative cost once you include transaction costs.

Before attempting general optimization frameworks, implement a first production-grade arbitrage checker as a set of hard-coded constraint families (complements, partitions, mutual exclusion) solved by small linear programs or even closed-form checks, using current order book best prices and the actual fee model.

Only iterate toward more general optimization once the simple version is stable and profitable in paper trading.

Execution Layer: Requirements Before Live Trading

For Polymarket, (This is the link if you want to register) you must incorporate order placement mechanics (marketable limit order behavior), the authentication structure (L1 EIP-712 signing and L2 HMAC credentials for trading requests), and fee model differences by market type.

For Kalshi, use the demo environment first with separate credentials from production, implement the documented authentication signing approach (API requests signed with specific headers), and ensure compliance with the Developer Agreement referenced in API documentation.

Critical execution correctness requirement: cross-market arbitrage only exists if you can get filled on all legs at the assumed prices. Your paper trader must therefore simulate fills against actual order book depth, not theoretical mid prices, and must include conservative slippage assumptions, partial fill handling (cancel remaining legs or hedge the orphaned position), and a strict "no orphan legs" rule where you never end up with one side of an arbitrage executed and the other side unfilled.

Paper Trading As An Engineering Acceptance Gate

Kalshi's demo environment is explicitly designed to test integrations with mock funds. For venues without formal demo environments, your simulator must be treated as your exchange of record for acceptance testing.

Minimum paper trading acceptance criteria:

Deterministic replay from logs (same inputs produce same outputs every time). Complete reconciliation where every simulated order maps to a simulated fill or cancel event. Profit and loss reporting with complete fee breakdown showing exactly where costs occurred. Tracking of missed

opportunities due to latency or execution delays. Complete removal of all secrets from logs (API keys and signatures never written to disk).

Only after your paper trading system passes all these criteria should you even consider connecting to live venues with real money.

Security And Governance For Unattended Operation

A brain-and-hands system that can read files, run commands, and place trades is not just a productivity tool. It's a privileged automation system. Treat it accordingly.

Reduce Blast Radius By Design

OpenClaw's workspace documentation explicitly warns that the workspace is the default working directory but not a hard sandbox unless sandboxing is specifically enabled.

Therefore: keep the workspace isolated (dedicated directory, no personal documents mixed in), run the agent under a dedicated user account or isolated host, and enable sandboxing for tool execution if your OpenClaw deployment supports it.

Lock Down Tools And Outbound Actions

OpenClaw documents global tool allow and deny controls, emphasizing that denied tools are not even sent to AI providers. Your hands should not have unrestricted shell or network access by default.

A workable policy set:

Allow: filesystem tools within workspace only, process execution limited to an explicit whitelist, HTTP access only to whitelisted API hosts (Kalshi and

Polymarket endpoints)

Deny: credential vault reads except runtime injection, arbitrary web browsing tools, package installation tools in production mode

Assume Third-Party Extensions Are Hostile

Recent reporting has highlighted malicious OpenClaw skills distributed through public registries, including malware specifically targeting crypto users, and warnings about misconfiguration leading to system breaches.

Engineering response: do not install community skills in the same runtime that holds trading credentials. Treat any skill as executable code requiring source review and version pinning. Separate your productivity agent and trading agent into completely distinct workspaces, profiles, and OS user accounts.

Secrets Management And Log Safety

OpenClaw documentation states that session transcripts are stored on disk and that filesystem access is the trust boundary, meaning any process or user with disk access can read them.

Accordingly: store exchange API keys outside the workspace in the state directory only with strict file permissions, inject secrets via environment variables rather than writing them into task files, and implement log scrubbing and redaction so headers, signatures, and private keys never appear in logs.

Compliance Constraints Are Requirements

Kalshi's API documentation explicitly binds API usage to the developer agreement. Polymarket's documentation describes authentication flows

involving cryptographic signing and credential derivation.

Operationally, a working system includes respecting rate limits, adhering to KYC and eligibility rules (never attempting to bypass them), and following venue terms for automated trading.

Reliability Guardrails For Live Execution

Even after successful paper trading, a production-grade bot requires a kill switch (one file drop like state/STOP that immediately halts new orders), maximum exposure rules (per market, per venue, per day limits), latency measurement with refusal to trade when data is stale, and safe mode activation on abnormal variance like sudden spread widening.

These are standard controls from automated trading operations. They're not optional if you want genuinely unattended operation under bounded risk.

Why This Architecture Actually Works

After building and testing this system, what makes it work isn't any single component. It's the combination of clear separation of concerns, explicit protocols, crash-safe file operations, and security boundaries that prevent small mistakes from becoming catastrophic failures.

The brain-and-hands architecture works because each layer does what it's good at. The brain provides judgment and planning without getting bogged down in execution details. The hands execute reliably without requiring high-level reasoning for every file operation.

The shared workspace folder works because it's deterministic. No hoping the AI remembers context. No praying that long-running conversations don't

drift. Just files, state machines, and atomic operations that survive crashes and restarts.

The prediction market trading layer works (when it works) because it's built on mathematical constraints that must hold, not probabilistic patterns that might hold. And even then, it only works if you properly incorporate fees, spreads, execution risk, and all the messy reality that separates theory from practice.

This isn't a weekend project. This is a properly engineered system that requires careful thought about error handling, security boundaries, state management, and operational safety. But once built correctly, it can run unattended for days, discovering opportunities, executing paper trades, and only escalating to humans when genuinely stuck on judgment calls.

The code you need is out there. The tools exist. The protocols work. What most guides are missing is the brutal honesty about what actually fails in production and how to design around those failure modes from the start.

Build the brain. Build the hands. Define the protocol. Test relentlessly in paper trading. Only then consider live execution. And always, always maintain the kill switch.

Affiliate Disclosure: *This article contains an affiliate link to Polymarket. If you choose to register through the provided link, I may earn a commission at no additional cost to you. This commission helps support the work behind this content and allows me to continue providing in-depth analysis and insights. Please note that any recommendations made are based on independent research and editorial judgment. The inclusion of an affiliate link does not influence the objectivity of the*

information presented. You are encouraged to conduct your own due diligence before making any financial or platform-related decisions.

Technology Cryptocurrency Money Data Science Programming

Published in Dev Genius

31K followers · Last published 8 hours ago

Follow

Coding, Tutorials, News, UX, UI and much more related to development

Written by Ezekiel Njuguna

787 followers · 149 following

Follow

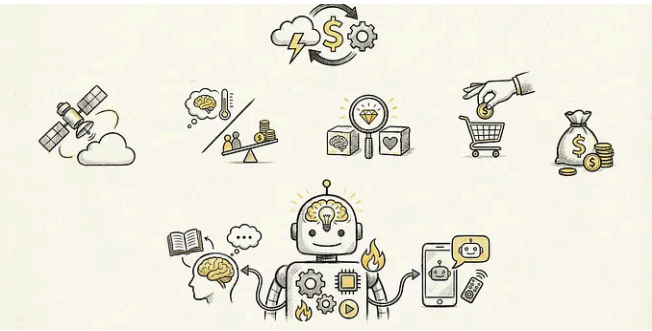
Writing is, and always will be, my thing. Let's work together:
ezzekielnjuguna.en@gmail.com

No responses yet

Write a response

What are your thoughts?

More from Ezekiel Njuguna and Dev Genius

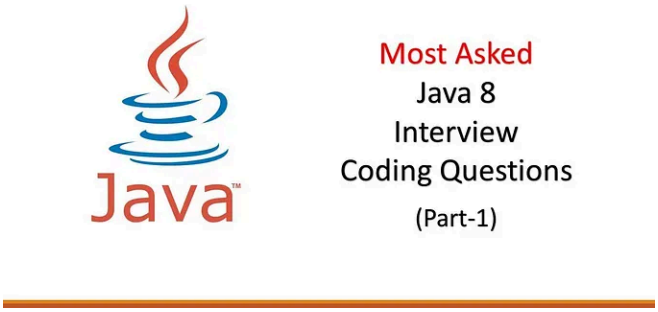


In Dev Genius by Ezekiel Njuguna

Found The Weather Trading Bots Quietly Making $24,000 On...

I want to tell you about something that’s been happening quietly on Polymarket for the last...

Feb 12 71 2



In Dev Genius by Anusha SP

Java 8 Coding and Programming Interview Questions and Answers

It has been 8 years since Java 8 was released. I have already shared the Java 8 Interview...

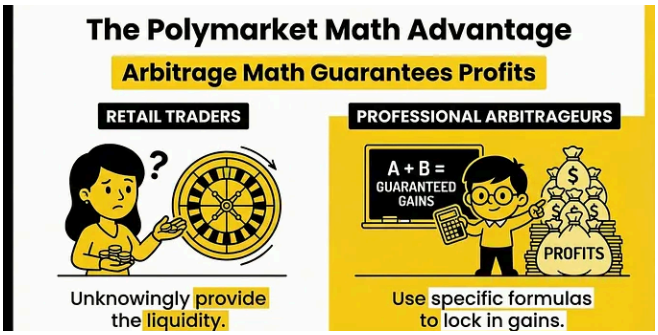
Jan 31, 2023 1.6K 23

In Dev Genius by Simra Husain

I Decompiled My Own Flutter APK. Here's Every Secret It Was Leaking.

I used one terminal command to extract API keys, endpoint URLs, and every class name...

Mar 23 95 2



In Dev Genius by Ezekiel Njuguna

Just Found the Math That Guarantees Profit on Polymarket...

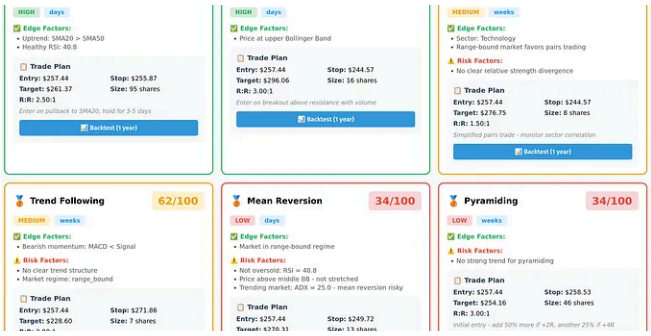
I am lost for words after what I discovered about Polymarket and the mathematical...

Feb 1 404 7

See all from Ezekiel Njuguna

See all from Dev Genius

Recommended from Medium



In DataDrivenInvestor by Pavel Zapolskii

Hunting for Mispriced Options: Building an Arbitrage Volatility Bo...

Or: How I Built a Robot to Collect Gold's Crumbs from Wall Street's Table

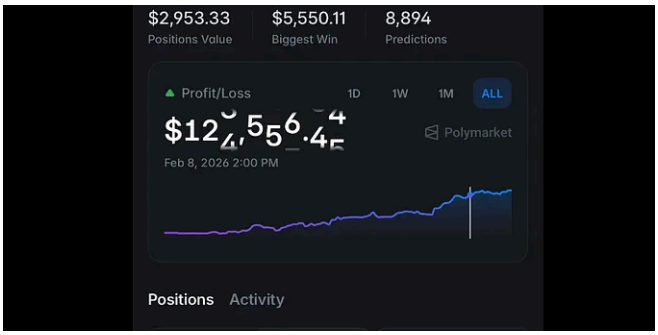
Feb 9 272 5

Minyang Chen

Explore with an AI-powered tool to automatically identify "best"...

There are many popular ways to trade stocks. To keep things simple, we can group the mo...

Mar 9 19 2

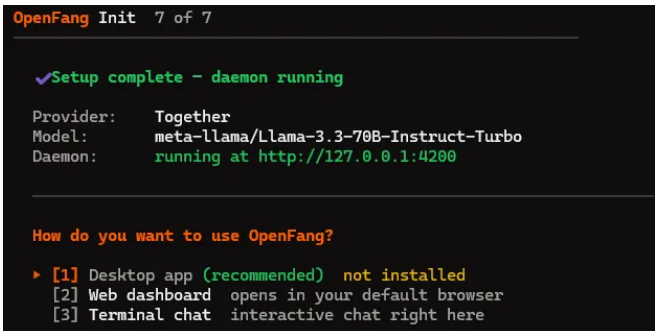


In Coding Nexus by Jatin Prasad

Don't Fall for the Viral Clawdbot Polymarket Arbitrage Setup

Before you run a dedicated machine and wire USDC into Polymarket, Read this

Feb 18 14 2



Agent Native

I Ignored 30+ OpenClaw Alternatives Until OpenFang

Fully open-source Agent Operating System, written entirely in Rust, shipping as a single...

Mar 10 655 16

In Curated Newsletters by Sips & Scale

How to Win Polymarket Bets Safely with and without Bots

Working strategies for Polymarket and Kalshi

In CoinsBench by Moonsat

Mar 16 64

Inside the Mind of a Polymarket BOT

If you have ever opened a Bitcoin 15-minute market on Polymarket and wondered why on...

Dec 4, 2025 135 116

See more recommendations